Java 8 (2014) - The Big Game Changer

✅ **Lambda Expressions** → Functional-style programming.
✅ **Streams API** → Efficient data processing (parallel streams).
✅ **Optional<T>** → Avoid `NullPointerException`.
✅ **Default Methods in Interfaces** → Interface evolution without breaking changes.
✅ **New Date & Time API** → `java.time` (Immutable & thread-safe).
✅ **Concurrent Improvements** → `CompletableFuture` for async programming.
✅ **Garbage Collector (GC) Enhancements** → G1GC as default.

Java 9 (2017) - Modularity & Performance

✅ **JPMS (Java Platform Module System)** → Better encapsulation.
✅ **JShell (REPL)** → Interactive Java execution.
✅ **Compact Strings** → Saves memory (UTF-16 vs Latin-1 storage).
✅ **Improved G1GC** → Low-latency improvements.
✅ **Immutable Collections** → `List.of(), Set.of(), Map.of()` factory methods.

Java 10 (2018) - Local Variable Inference

✅ **`var` keyword** → Simplifies local variable declarations.
✅ **G1GC Parallel Full GC** → Better performance.

Java 11 (2018) - LTS (Long-Term Support)

✅ **ZGC (Experimental)** → Ultra-low pause time GC.
✅ **Removed Java EE & CORBA** → Lighter JDK.
✅ **New String Methods** → `isBlank(), lines(), repeat()`.

Java 12-14 (2019-2020) - Performance & Preview Features

✅ **Switch Expressions (Java 14)** → Concise switch syntax.
✅ **Shenandoah GC (Java 12)** → Low-pause GC.
✅ **Pattern Matching (Preview in 14)** → `instanceof` simplification.

Java 15-16 (2020-2021) - Modernization

✅ **Records (Java 15)** → Immutable, concise data classes.
✅ **Sealed Classes (Java 15)** → Restrict class inheritance.
✅ **ZGC (Java 15, Production-Ready)** → Low-latency GC.
✅ **Foreign Function API (Incubator in 16)** → Call native code efficiently.

Java 17 (2021) - LTS & Better Performance

✅ **Pattern Matching for Switch** (Finalized).
✅ **Sealed Classes (Finalized)** → Controlled inheritance.
✅ **Removed RMI Activation System** (Legacy cleanup).
✅ **Improved Performance** → Optimized memory footprint.

Java 18-19 (2022) - Performance & GC Tuning

✅ **UTF-8 as Default Charset (Java 18)**.
✅ **Virtual Threads (Java 19, Preview)** → Lightweight, scalable concurrency (Project Loom).
✅ **Structured Concurrency API (Java 19, Preview)** → Simplifies thread management.

Java 20-21 (2023) - Major LTS Release!

✅ **Virtual Threads (Finalized in Java 21)** → Millions of threads with low overhead.
✅ **Record Patterns (Java 21)** → Pattern matching for records.
✅ **Shenandoah GC Optimizations (Java 21)** → Further reduced pause times.
✅ **Sequenced Collections (Java 21)** → Ordered collections with new APIs.
✅ **Scoped Values (Incubator in 21)** → More efficient thread-local data sharing.

🔥 Key Takeaways

1️⃣ **Java 8** → Lambdas, Streams, CompletableFuture.
2️⃣ **Java 9-10** → JPMS (Modules), `var` keyword.
3️⃣ **Java 11 (LTS)** → Performance, GC updates.
4️⃣ **Java 17 (LTS)** → Pattern Matching, Sealed Classes.
5️⃣ **Java 21 (LTS)** → **Virtual Threads**, Record Patterns, Sequenced Collections.

1️⃣ Virtual Threads (Project Loom) - Java 19/21

- 🔹 **Traditional Threads (Platform Threads)** → OS-managed, heavyweight (~1MB stack).
- 🔹 **Virtual Threads (Java 19 - Preview, Java 21 - Finalized)** → JVM-managed, lightweight (KBs of stack).

✨ Why Virtual Threads?

✔ **Millions of threads instead of thousands** (lightweight, low overhead).
✔ **No thread pool bottlenecks** (1:1 mapping removed).
✔ **Non-blocking I/O operations scale better**.

🚀 Example: Virtual Threads vs Platform Threads

```
// Virtual Threads (Java 21)
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i ->
        executor.submit(() -> {
            Thread.sleep(1000); // Simulates blocking I/O
            System.out.println("Task " + i);
        })
    );
} // Executor auto-closes
```

✅ **Spawns 10,000+ threads easily** (vs. traditional thread pools).

2️⃣ G1GC (Garbage-First Garbage Collector) - Optimizations

🔹 **G1GC (Default since Java 9, optimized in later versions) → Low-pause, concurrent GC**.

🔹 **Improvements in Java 17+ → Better NUMA handling, improved heap sizing, and optimized region management**.

🚀 Key JVM Tuning Flags for G1GC

```
-XX:+UseG1GC                      # Enable G1GC (Default from Java 9)
-XX:InitiatingHeapOccupancyPercent=40  # Start GC when 40% heap is used (default: 45%)
-XX:MaxGCPauseMillis=200          # Target max GC pause time
-XX:+ParallelRefProcEnabled       # Parallelize reference processing
-XX:G1HeapRegionSize=8M           # Tune region size (depends on heap)
```

✅ **Fine-tuning helps balance latency and throughput.**

3️⃣ Modern Java Concurrency Enhancements

- 🔹 **Structured Concurrency (Java 19-21 Preview)** → Organizes thread lifecycles better.
- 🔹 **Scoped Values (Java 21 Preview)** → Better than `ThreadLocal` for context-sharing.
- 🔹 **Pattern Matching + Switch Enhancements (Java 17-21)** → Cleaner concurrent code.

🚀 Example: Structured Concurrency

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Future<Integer> result1 = scope.fork(() -> computeA());
    Future<Integer> result2 = scope.fork(() -> computeB());
    scope.join();  // Wait for both tasks
    return result1.get() + result2.get();
}
```

✅ **Ensures proper thread shutdown, better error handling.**